

SINCRONIZACIÓN DE PROCESOS

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN

UNIVERSIDAD NACIONAL DEL SUR

KMC © 2026

Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Agenda

1. **Fundamentos**
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos



Fundamentos

- El acceso concurrente a datos compartidos puede resultar en inconsistencias.
- Mantener la consistencia de datos requiere mecanismos para asegurar la ejecución ordenada de procesos cooperativos.

Concurrencia



Multiprocesadores

Fundamentos

- Caso de análisis: problema del buffer limitado. Una solución, donde todos los N buffers son usados, no es simple.
  - Considere la siguiente solución

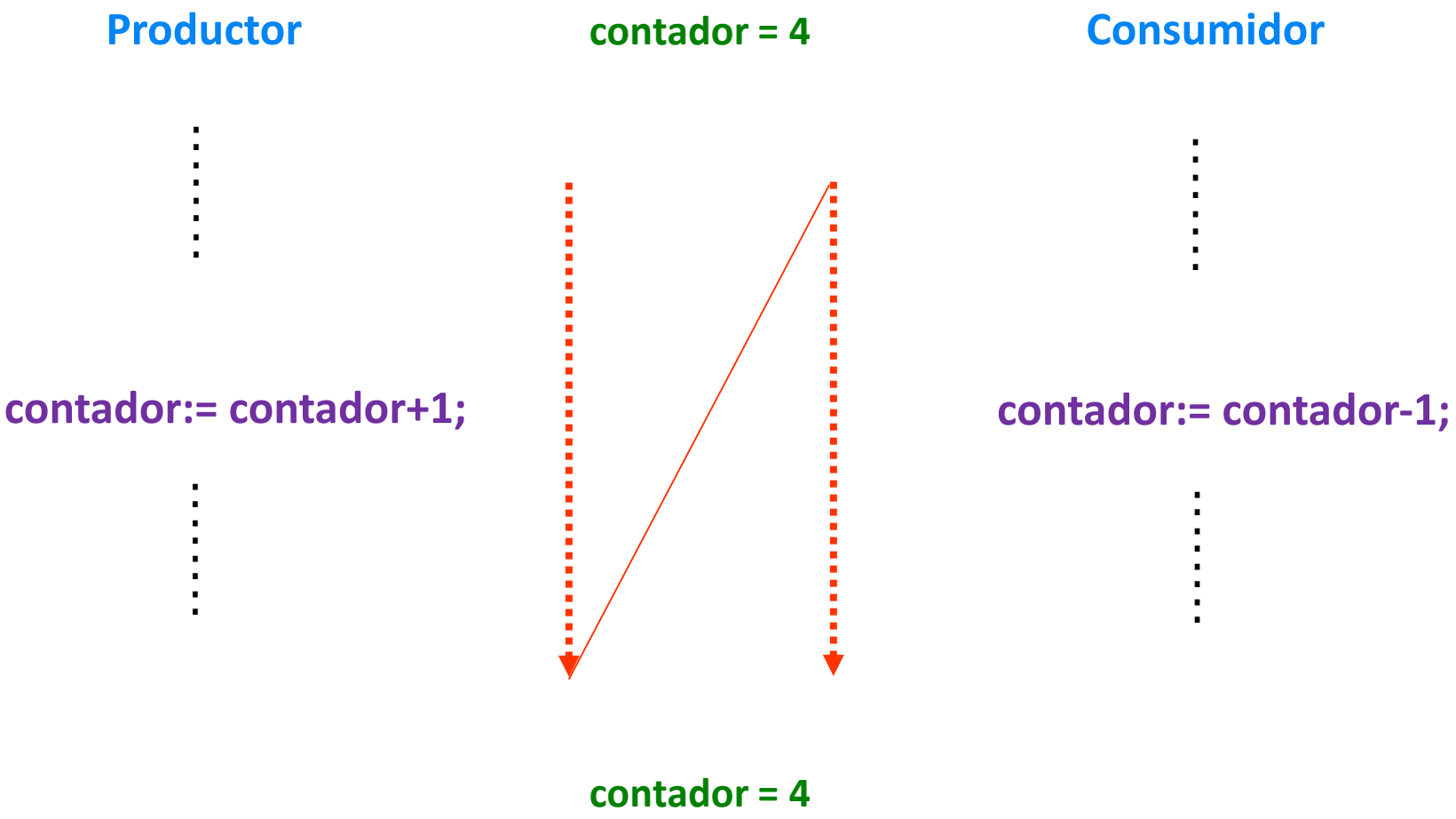
DATOS COMPARTIDOS

```
type item = … ;
var buffer array [0..n-1] of item;
in, out: 0..n-1;
contador : 0..n;
in, out, contador := 0;
```

Productor-Consumidor

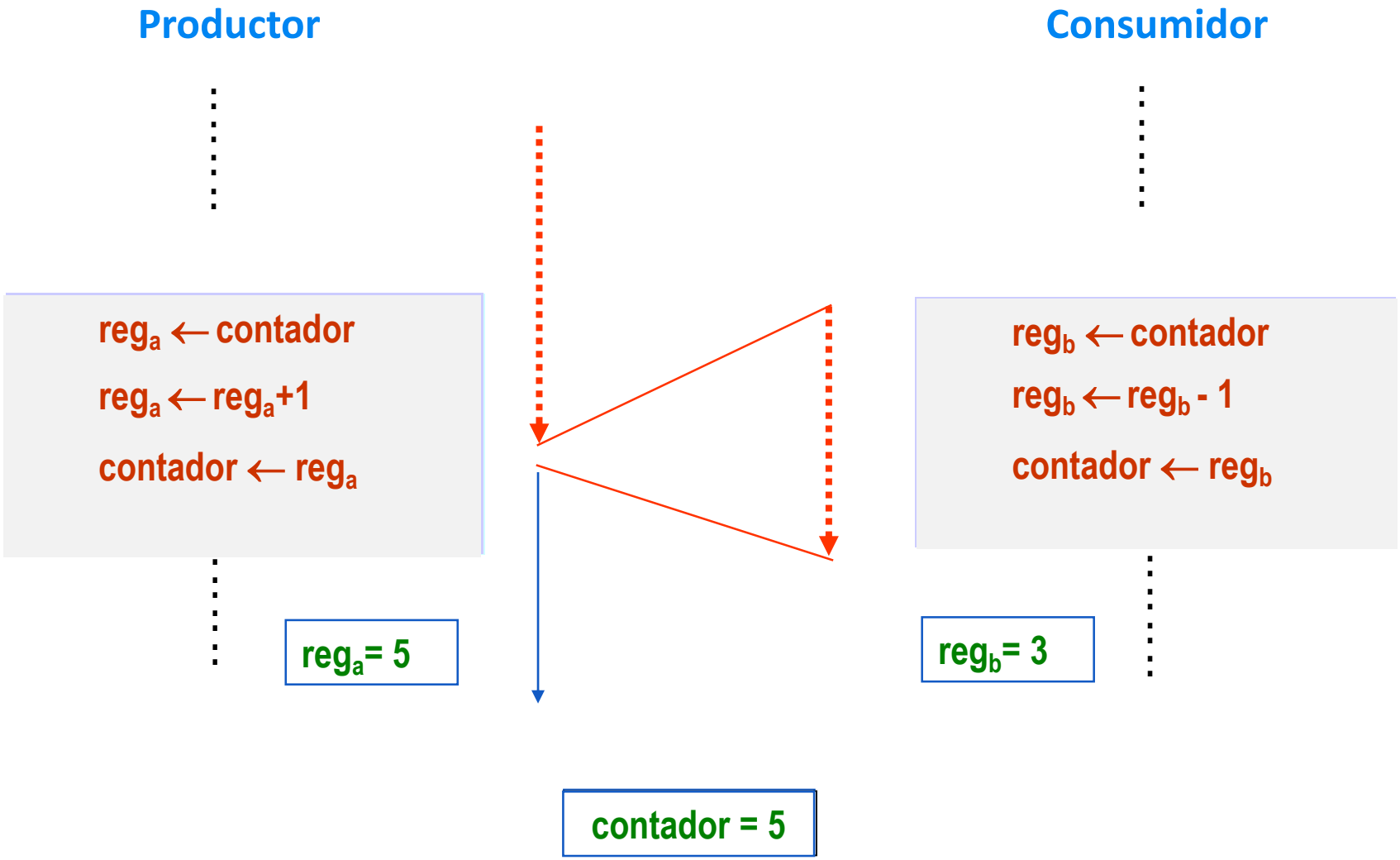
Proceso Productor	Proceso Consumidor
repeat ... produce un item en *nextp* ... **while** *contador* = n **do** no-op; *buffer* [*in*] := *nextp*; *in* := *in* + 1 **mod** *n*; *contador* := *contador* +1; **until** false;	**repeat** **while** *contador* = 0 **do** *no-op*; *nextc* := *buffer* [*out*]; *out* := *out* + 1 **mod** *n*; *contador* := *contador* – 1; ... consume el item en *nextc* ... **until** *false*;

Problema



Luego de una operación de Productor y otra de Consumidor …
contador permanece invariante

Problema

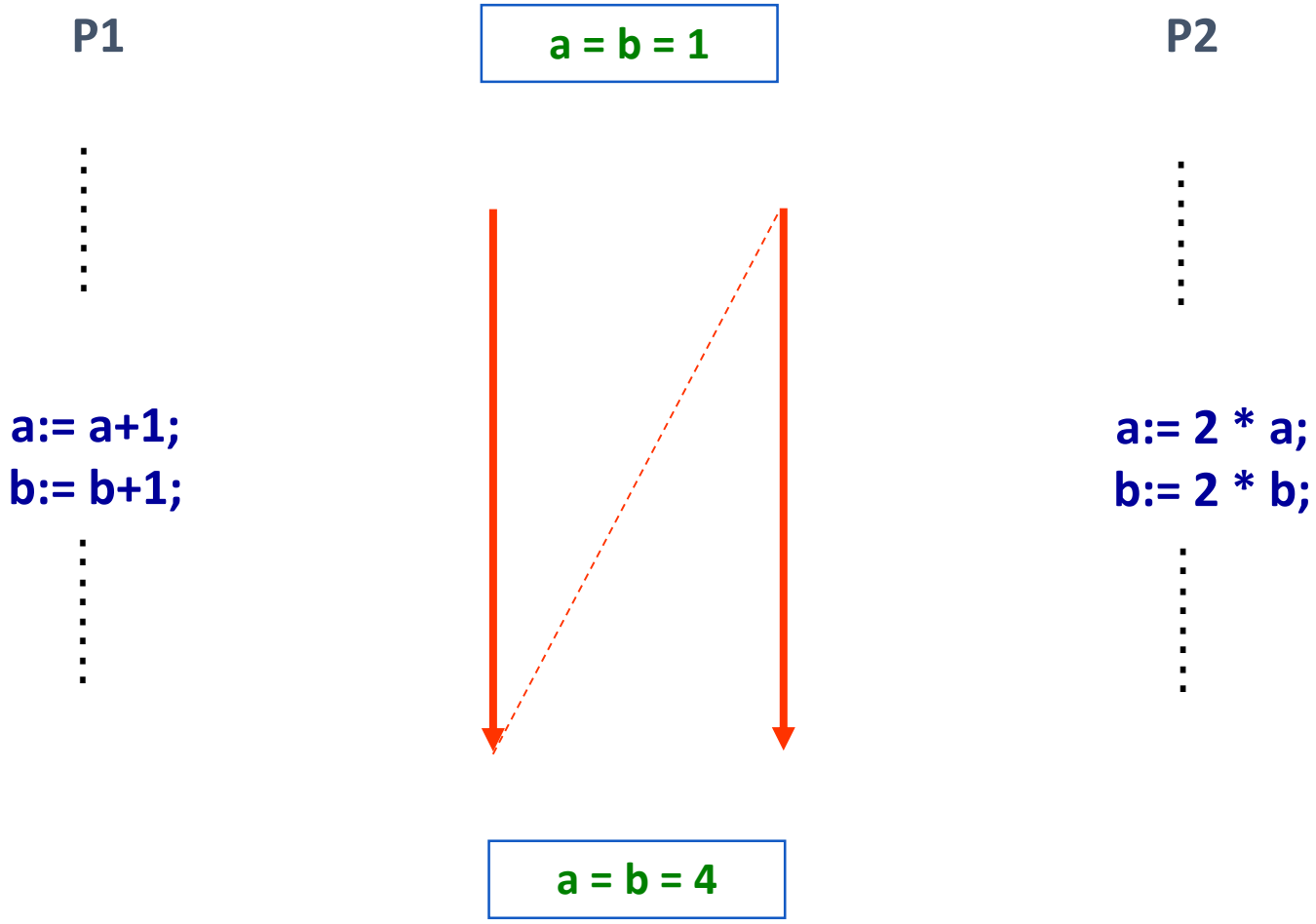

Productor
Consumidor
reg_a ← contador
reg_a ← reg_a+1
contador ← reg_a
reg_b ← contador
reg_b ← reg_b - 1
contador ← reg_b
reg_a= 5
reg_b= 3
contador = 5

Problema

Situaciones dónde puede ocurrir este problema.

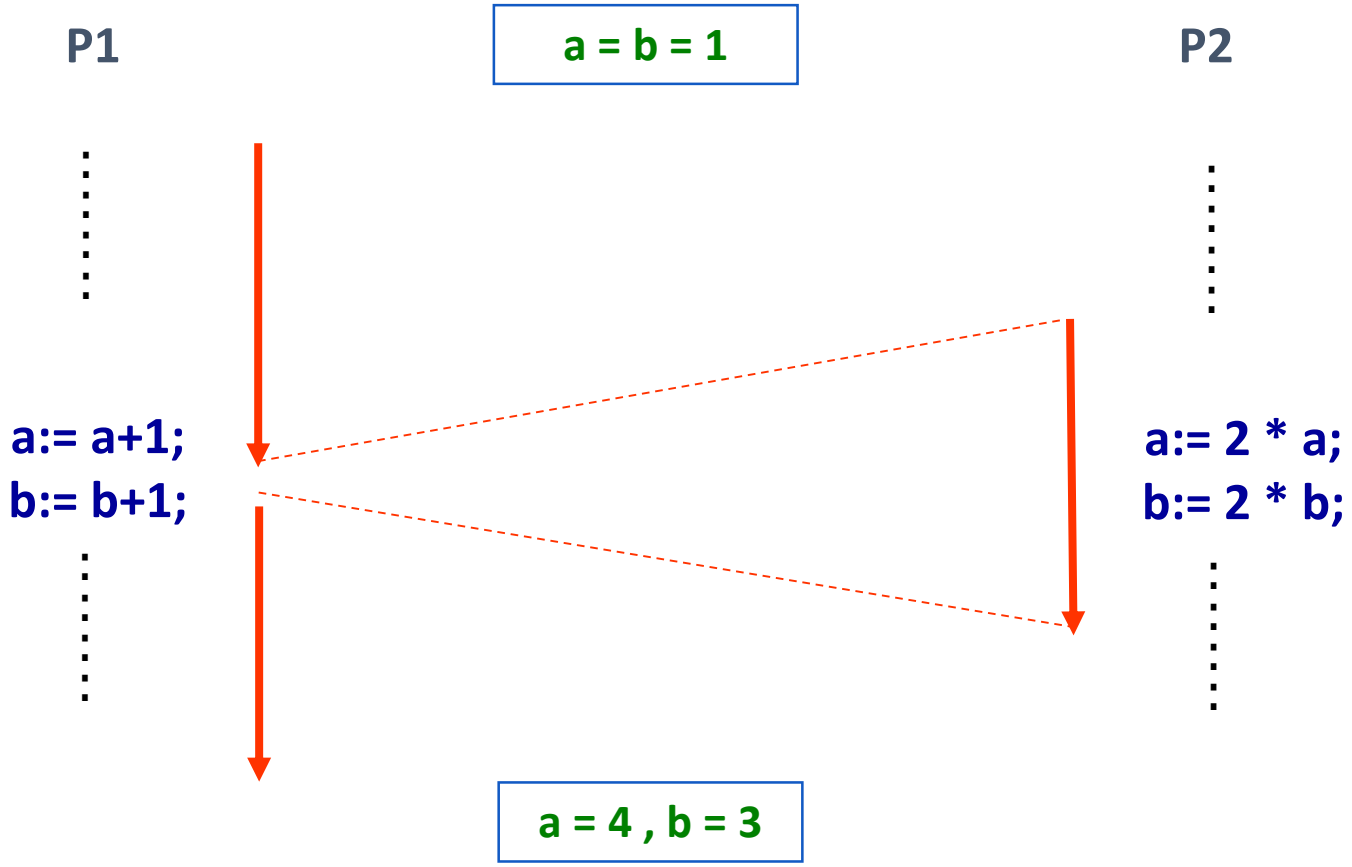
- Utilizando una variable compartida
- Ejecutando un conjunto de sentencias sobre variables compartidas
- Actualizando una estructura de datos, como por ejemplo una lista enlazada

Problema en conjunto de sentencias



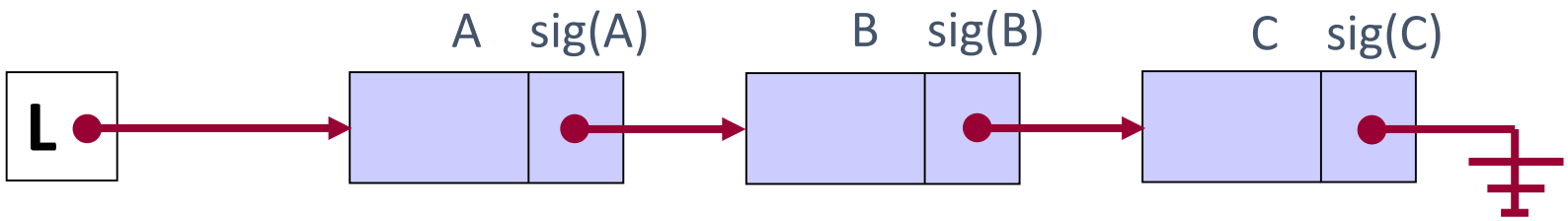
Luego de la ejecución de P1 seguida de P2 los valores de *a* y *b* deben ser iguales entre sí

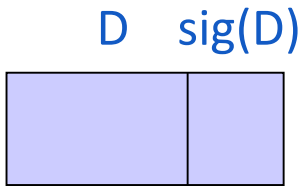
Problema en conjunto de sentencias



Luego de la ejecución de P1 y P2 los valores de *a* y *b* deberían ser iguales entre sí, *pero no lo son.*

Problema en estructura de datos



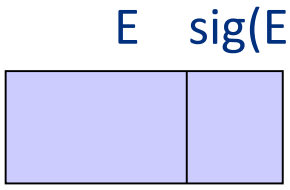


:

sig(D) ← L

L ← D

:



:

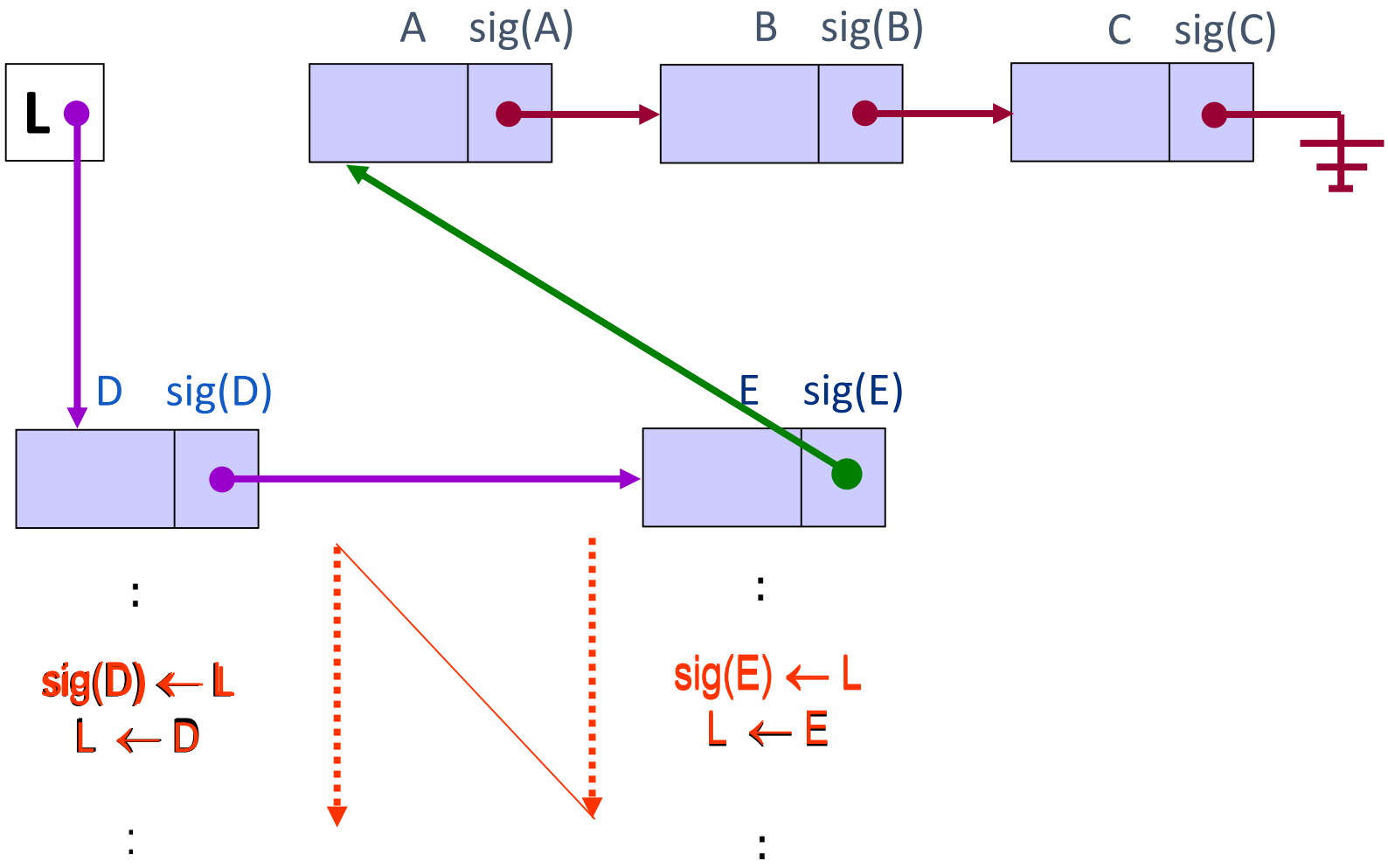
sig(E) ← L

L ← E

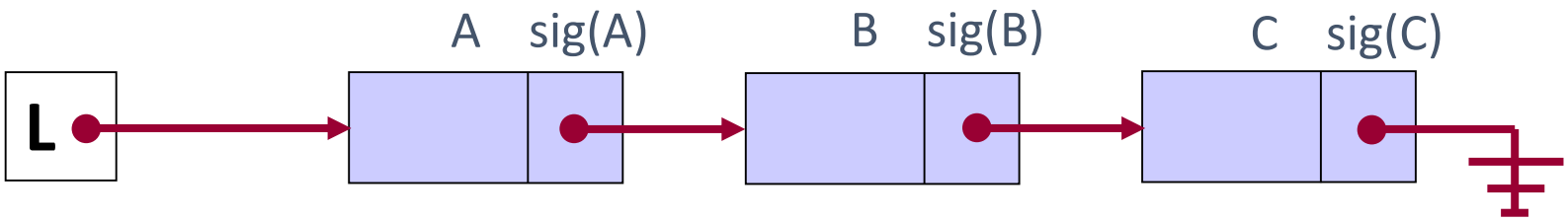
:

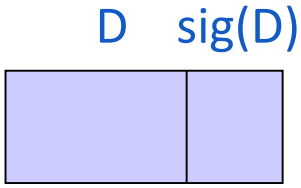
Por ejemplo:
Cola de listos

Problema en estructura de datos



Problema en estructura de datos



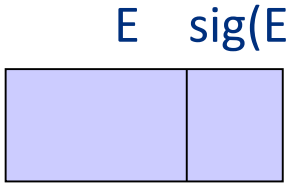


:

sig(D) ← L

L ← D

:



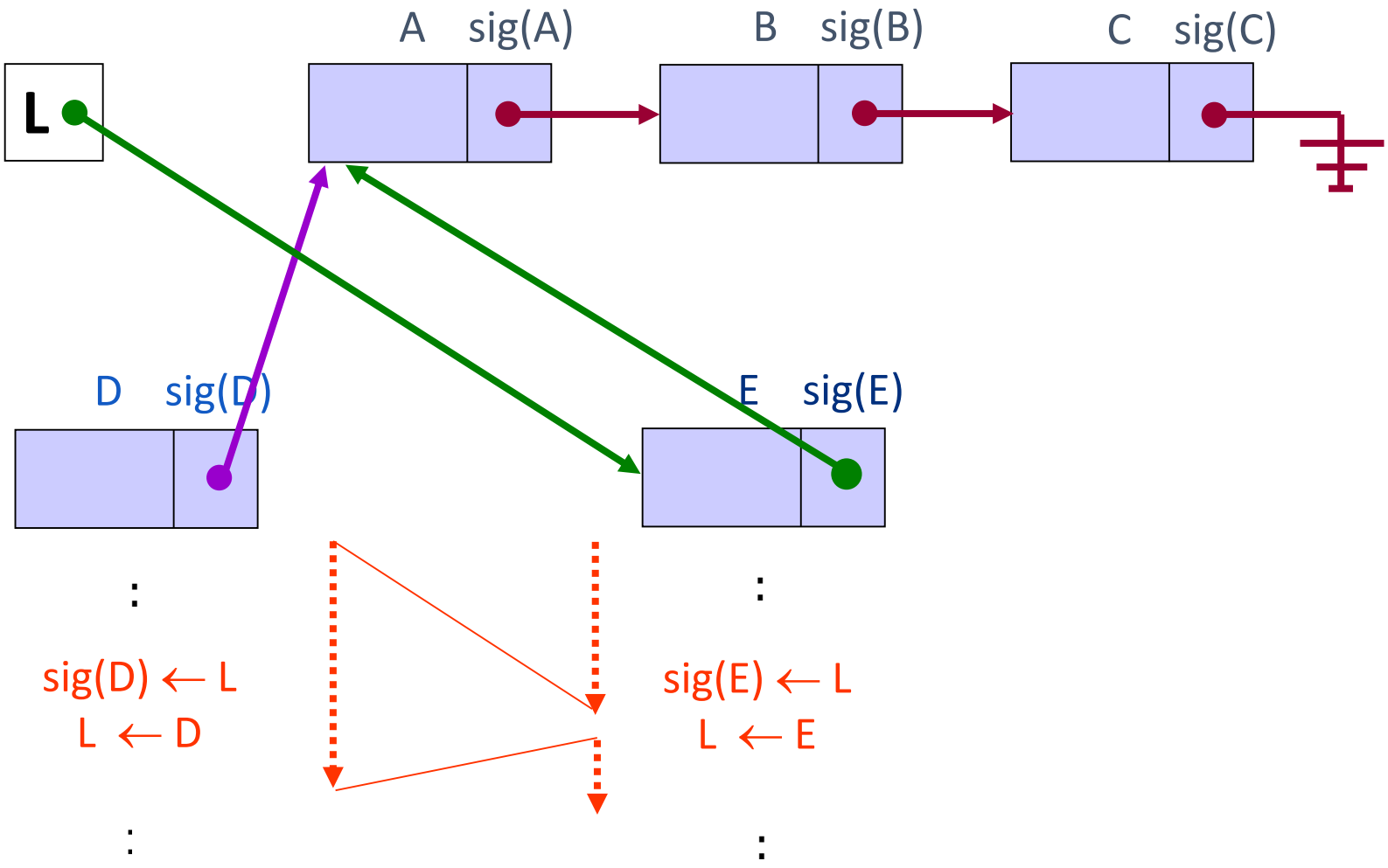
:

sig(E) ← L

L ← E

:

Problema en estructura de datos



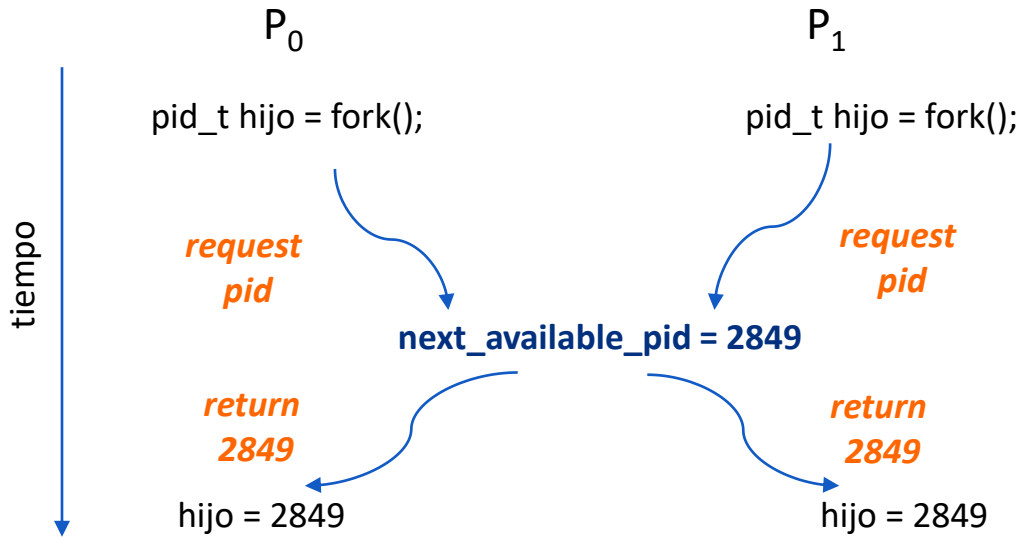
Problema

¿QUÉ OCURRE CON EL KERNEL?



Problema

- Procesos P_0 y P_1 están creando un proceso hijo utilizando la llamada al sistema **fork()**.
- El kernel tiene la variable **next_available_pid** la cual representa el próximo identificador disponible para un proceso (pid).



- Podría ocurrir que el mismo pid sea asignado a dos procesos diferentes.

Condición de Carrera

- **Condición de carrera**

  Es la situación donde varios procesos acceden y manejan datos compartidos concurrentemente. El valor final de los datos compartidos depende de que proceso termina último.

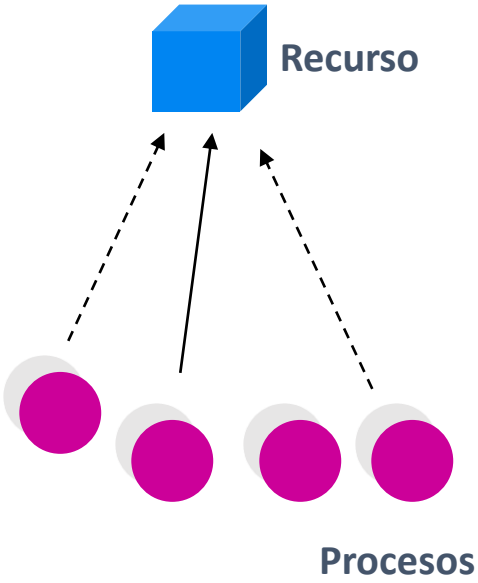
- Para prevenir las condiciones de carrera, los procesos concurrentes cooperativos deben ser **sincronizados**.

Agenda

1. Fundamentos
2. **El Problema de la Sección Crítica**
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Problema de la Sección Crítica

- n procesos todos compitiendo para usar datos compartidos
- Cada proceso tiene un segmento de código llamado ***sección crítica***, en la cual los datos compartidos son accedidos
- Problema – asegurar que cuando un proceso está ejecutando en su sección crítica, no se le permite a otro proceso ejecutar en su respectiva sección crítica.



Solución al Problema de la Sección Crítica

CONDICIONES PARA UN BUEN ALGORITMO

1. **Exclusión Mutua**
2. **Progreso**
3. **Espera Limitada**

- Asuma que cada proceso ejecuta a velocidad distinta de cero.
- No se asume nada respecto a la velocidad relativa de los n procesos.

Resolución del Problema

Estructura general del proceso P_i

repeat

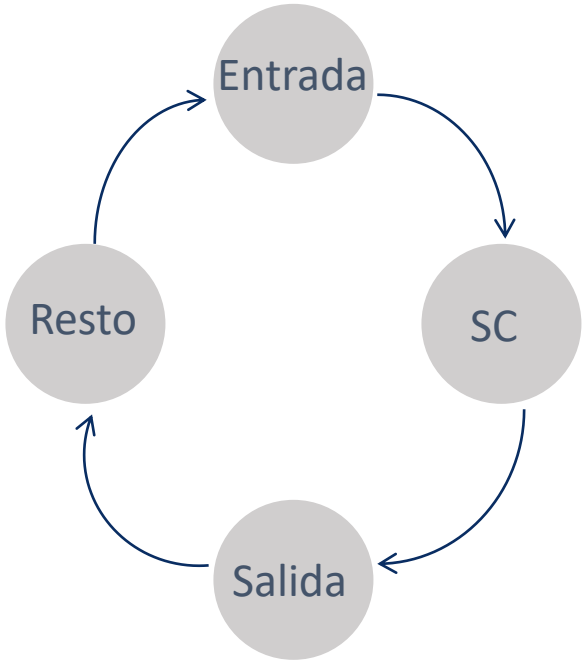
protocolo de entrada

sección crítica (SC)

protocolo de salida

sección resto

until *falso*



- Los procesos pueden compartir algunas variables comunes para sincronizar sus acciones.

Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. **Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)**
4. Soluciones por Hardware
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Solución para 2 procesos: Algoritmo de Peterson

DATOS COMPARTIDOS

int turno;

boolean flag[2]; incializado en ***false***

Proceso P_i

repeat

flag [*i*] := *true*;
turno := *j*;
while (*flag* [*j*] and *turno* = *j*) **do** *no-op*;

sección crítica

flag [*i*] := *false*;

sección resto

until *false*;

- Alcanza las propiedades de un buen algoritmo; resuelve el problema de la sección crítica para dos procesos.

Algoritmo para N Procesos – Algoritmo del Panadero

Sección crítica para *n* procesos

- Antes de entrar en su sección crítica, el proceso recibe un número. El poseedor del número más chico entra en su sección crítica.
- Si los procesos P_i y P_j reciben el mismo número, si i < j, entonces P_i es atendido primero; sino lo es P_j .
- El esquema de numeración siempre genera números en orden incremental de enumeración;

  p.e., 1,2,3,3,3,3,4,5...

Algoritmo para N Procesos – Algoritmo del Panadero

Notación orden lexicográfico (ticket #,id proceso #)

- (a,b) < (c,d) si a < c o si a = c y b < d
- max (a0,…, an-1) es un número k, tal que k ≥ ai para i = 0, …, n – 1

DATOS COMPARTIDOS

var *eligiendo*: **array** [0..*n* – 1] **of** *boolean*;

numero: **array** [0..*n* – 1] **of** *integer*,

Las estructuras de datos son inicializadas a ***false*** y 0 respectivamente.

Algoritmo para N Procesos – Algoritmo del Panadero

```
repeat
    eligiendo[i] := true;
    numero[i] := max(numero[0], numero[1], …, numero [n – 1])+1;
    eligiendo[i] := false;
    for j := 0 to n – 1
        do begin
            while eligiendo[j] do no-op;
            while numero[j] ≠ 0
                and (numero[j],j) < (numero[i], i) do no-op;
        end;
        sección crítica
    numero[i] := 0;
        sección resto
until false;
```

Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. **Soluciones por Hardware**
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Sincronización por Hardware

- Monoprocesador – pueden deshabilitarse las interrupciones

- Las computadoras modernas proveen instrucciones especiales que se ejecutan atómicamente

Atómico = no-interrumpible

  - Por verificación de una palabra de memoria y su inicialización. Por ejemplo: ***test-and-set***
  - Por intercambio de dos palabras de memoria. Por ejemplo: **swap**

Exclusión Mutua con Test-and-Set

DATO COMPARTIDO

var *lock*: *boolean* (*inicialmente* ***false***)

Proceso P_i

repeat

while *Test-and-Set* (*lock*) **do** *no-op*;

sección crítica

lock := *false*;

sección resto

until *false*;

```
función  Test-and-Set (var target: boolean): boolean;
         begin
              Test-and-Set := target;
              target := true;
         end;
```

Exclusión Mutua con Swap

La variable booleana compartida es inicializada en ***false***. Cada proceso tiene una variable local booleana *key*

Solución:

```
while (true)  {
        key = true;
        while ( key == true)
                Swap (&lock, &key );

                sección crítica

        lock = false;

                sección restante

}
```

```
void Swap (boolean *a, boolean *b)
        {
                boolean temp = *a;
                *a = *b;
                *b = temp:
        }
```

Soluciones por Hardware



Las soluciones presentadas utilizando *Test-and-Set* y *Swap* ¿son buenas soluciones para el problema de la sección crítica?

Bibliografía:

- Silberschatz, A., Gagne G., y Galvin, P.B.; "Operating System Concepts", 7ma Edición 2009, 9na Edición 2012, 10ma Edición 2018.
- Stallings, W. "Operating Systems: Internals and Design Principles", Prentice Hall, 7ma Edición 2011, 8va Edición 2014, 9na Edición 2018.
- Tanenbaum, A.; "Modern Operating Systems", Pearson, 3rd Edición 2008, 4th Edición 2014, 5th. Edition 2022.